

Heap Sort

Funcionamento

O **Heap Sort** utiliza uma estrutura chamada **Heap**, normalmente um **Max-Heap** para ordenar em ordem crescente.

Max-Heap

No Max-Heap, o pai sempre é **maior ou igual** aos seus filhos. Para um vetor com índice começando em 0:

```
Pai:      (i - 1) / 2
Esquerda: 2 * i + 1
Direita:  2 * i + 2
```

Passos

1. **Construir o Max-Heap** a partir do vetor.
2. O maior elemento fica na **raiz** (**vetor[0]**).
3. Trocar a raiz com o último elemento da parte não ordenada.
4. Reduzir o tamanho do Heap.
5. Reorganizar o Heap usando **heapify**.
6. Repetir até o vetor estar ordenado.

```
Vetor inicial
  ↓
Construir Max-Heap
  ↓
Maior elemento → final do vetor
  ↓
heapify novamente
  ↓
Repetir
  ↓
Vetor ordenado
```

Complexidade

Observação: a construção inicial do Heap é $O(n)$, mas as remoções/reorganizações fazem o algoritmo completo ficar $O(n \log n)$.

Estabilidade

Heap Sort é INSTÁVEL. Elementos com valores iguais podem trocar de posição durante as operações do Heap.

Características

Característica	Heap Sort
Método	Comparação
Estrutura usada	Heap
Melhor	$O(n \log n)$
Médio	$O(n \log n)$
Pior	$O(n \log n)$
Memória	$O(1)$
Estável?	Não
In-place?	Sim

Implementação em C++ — sem STL

```
void heapify(int v[], int n, int i) {
    int maior = i;
    int esquerda = 2 * i + 1;
    int direita = 2 * i + 2;
    if (esquerda < n && v[esquerda] > v[maior])
        maior = esquerda;
    if (direita < n && v[direita] > v[maior])
        maior = direita;
    if (maior != i) {
        int temp = v[i];
        v[i] = v[maior];
        v[maior] = temp;
        heapify(v, n, maior);
    }
}

void heapSort(int v[], int n) {
    // Constrói o Max-Heap
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(v, n, i);
    // Coloca o maior elemento no final
    for (int i = n - 1; i > 0; i--) {
        int temp = v[0];
        v[0] = v[i];
        v[i] = temp;
        heapify(v, i, 0);
    }
}
```